

# C String Processing

Grounded in ECM2433 arrays, strings, and worksheet material. Focuses on null terminators, string storage, standard library functions, bounded copying, character classification, in-place editing, and common trace/fix tasks.

## What You Need To Know

### 1. C strings are null-terminated char arrays

A C string is a sequence of characters stored in an array and ended by `'\0'`. That terminator is how `printf`, `strlen`, and the string library know where the text stops. If the terminator is missing, string code can keep reading beyond the intended array.

```
char s[] = "Hello";
/* stored as: H e l l o \0 */

printf("%s\n", s);    /* Hello */
s[2] = '\0';
printf("%s\n", s);    /* He */
```

### 2. sizeof and strlen answer different questions

`sizeof` measures storage in bytes. `strlen` counts characters before the first `'\0'`. Changing the contents of an array can change `strlen`, but not the array's `sizeof`.

```
char text[20] = "Hello world";

sizeof(text)    /* 20: total array storage */
strlen(text)    /* 11: characters before \0 */

text[0] = '\0';
sizeof(text)    /* still 20 */
strlen(text)    /* now 0 */
```

### 3. Printing strings with width and precision

For `printf` string formatting, width sets the minimum field width. Precision limits the maximum number of characters printed. A minus sign left-aligns the output.

```
char name[] = "Elephants";

printf("%s\n",      name);    /* *Elephants* */
printf("%20s\n",    name);    /* right-aligned in width 20 */
printf("%-20s\n",   name);    /* left-aligned in width 20 */
printf("%-20.3s\n", name);    /* prints only Ele, then pads */
```

### 4. Standard library string functions

Include `<string.h>` for the standard string functions. They all expect valid null-terminated strings.

- `strlen(s)`: counts characters before `'\0'`.
- `strcmp(a, b)`: returns 0 when the two strings have the same contents.
- `strcpy(dest, src)`: copies `src` into `dest`, but does not know the size of `dest`.
- `strncpy(dest, src, n)`: copies up to `n` characters, but may not add a null terminator if `src` is too long.
- `snprintf(buf, size, fmt, ...)`: writes formatted text into a bounded buffer.

```
char dest[8];
strncpy(dest, src, sizeof dest - 1);
dest[sizeof dest - 1] = '\0';
```

### 5. Character classification and conversion

Include `<ctype.h>` for functions such as `isalpha`, `isdigit`, `ispunct`, and `tolower`. These are useful when normalising text before comparing or searching it.

```
for (int i = 0; s[i] != '\0'; i++) {
    if (isalpha((unsigned char)s[i]))
        s[i] = tolower((unsigned char)s[i]);
}
```

### 6. In-place string editing

A common pattern is to read from one index and write to another. The read index visits every original character. The write index only advances when a character is kept. At the end, add a new null terminator.

```
void remove_spaces(char s[]) {
```

```

    int write = 0;
    for (int read = 0; s[read] != '\0'; read++) {
        if (s[read] != ' ')
            s[write++] = s[read];
    }
    s[write] = '\0';
}

```

## 7. Palindrome checks use mirrored indexes

To compare the first character with the last, use `s[ len - 1 - i]`. The minus 1 matters because `s[ len]` is the null terminator, not the final visible character.

```

bool is_palindrome(char s[]) {
    int len = strlen(s);
    for (int i = 0; i < len / 2; i++) {
        if (s[i] != s[len - 1 - i])
            return false;
    }
    return true;
}

```

## Practice Questions

Questions 1 and 2 use this declaration:

```
char s[20] = "Hello world";
```

- 1 Calculate `sizeof(s)` and `strlen(s)`.
- 2 After `s[0] = '\0'`, calculate `sizeof(s)` and `strlen(s)` again.

Question 3 uses this code:

```

char name[] = "Elephants";
printf("%*-20.3s*\n", name);

```

- 3 Write the exact output, including the asterisks and spaces.

Question 4 uses this code:

```

char text[] = "abc123!";
int letters = 0, digits = 0, other = 0;

for (int i = 0; text[i] != '\0'; i++) {
    if (isalpha((unsigned char)text[i]))
        letters++;
    else if (isdigit((unsigned char)text[i]))
        digits++;
    else
        other++;
}

printf("%d %d %d\n", letters, digits, other);

```

- 4 Trace the loop and write what the program prints.

Question 5: complete the function body.

```

void strip_digits(const char *src, char *dst) {
    int write = 0;
    for (int read = 0; src[read] != '\0'; read++) {
        /* complete this */
    }
    /* complete this */
}

```

- 5 Complete the function so it copies `src` to `dst` while skipping digit characters.

Question 6: complete the missing expression.

```

bool is_palindrome(char s[]) {
    int len = strlen(s);
    for (int i = 0; i < len / 2; i++) {
        if (s[i] != s[_____])
            return false;
    }
    return true;
}

```

- 6 Fill the blank with the mirrored index expression.

Question 7: complete the function body.

```

void normalise(char s[]) {
    int write = 0;
    for (int read = 0; s[read] != '\0'; read++) {
        /* keep letters/digits, remove punctuation/spaces, lowercase kept chars */
    }
    /* finish the string */
}

```

- 7 Complete the function and state the header needed for the character functions.

Question 8 uses this code:

```

char word[] = "racecar";
if (is_palindrome(word)) {
    printf("yes\n");
} else {
    printf("no\n");
}

```

- 8 Trace the condition and write what is printed.

Question 9: complete the function.

```

size_t mystrlen(const char *s) {
    size_t n = 0;
    /* complete this */
    return n;
}

```

- 9 Complete mystrlen without using any string library function.

Question 10: complete the bounded copy.

```

void string_copy(char *dest, size_t len, const char *src) {
    size_t i = 0;
    /* copy at most len - 1 characters */
    /* always add a null terminator */
}

```

- 10 Complete the function so it works for a destination buffer of size len.

# Mark Scheme

Attempt the questions before checking. Equivalent code is fine if it has the same behaviour and still null-terminates strings correctly.

- 1 `sizeof(s)` is 20. `strlen(s)` is 11.
- 2 `sizeof(s)` is still 20. `strlen(s)` is 0 because the first character is now the null terminator.
- 3 It prints an opening asterisk, Ele, 17 spaces, then a closing asterisk. The precision prints 3 characters, then the width pads to 20 characters.
- 4 It prints 3 3 1: three letters, three digits, and one other character.
- 5 Inside the loop, copy only non-digits: `if (!isdigit((unsigned char)src[read])) dst[write++] = src[read];`. After the loop, write `dst[write] = '\0';`.
- 6 Use `len - 1 - i`. The last visible character is at index `len - 1`.
- 7 Include `<ctype.h>`. Keep only characters where `isalnum((unsigned char)s[read])` is true, store `tolower((unsigned char)s[read])`, and finish with `s[write] = '\0';`.
- 8 It prints yes because racecar reads the same forwards and backwards.
- 9 Loop while `s[n] != '\0'`, incrementing `n`. Return `n`.
- 10 Loop while `i + 1 < len` and `src[i] != '\0'`, copying `src[i]` into `dest[i]`. Then set `dest[i] = '\0';`.

## Useful Code Patterns

```
#include <ctype.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>

size_t mystrlen(const char *s) {
    size_t n = 0;
    while (s[n] != '\0')
        n++;
    return n;
}

void string_copy(char *dest, size_t len, const char *src) {
    size_t i = 0;
    if (len == 0)
        return;
    while (i + 1 < len && src[i] != '\0') {
        dest[i] = src[i];
        i++;
    }
    dest[i] = '\0';
}

void normalise(char s[]) {
    int write = 0;
    for (int read = 0; s[read] != '\0'; read++) {
        unsigned char ch = (unsigned char)s[read];
        if (isalnum(ch))
            s[write++] = (char)tolower(ch);
    }
    s[write] = '\0';
}

bool is_palindrome(char s[]) {
    int len = strlen(s);
    for (int i = 0; i < len / 2; i++) {
        if (s[i] != s[len - 1 - i])
            return false;
    }
    return true;
}
```